

Ilg'or rekurrent neyron tarmoq arxitekturalari: GRU va LSTM

Tabiiy tilni qayta ishlash — 8-ma'ruza

A.A. Abdulali

11:00–12:20 · mavzu №8

26-iyun 2026

Bugun: xotirali ikki arxitektura

GRU = *Gated Recurrent Unit* (darvozali rekurrent birlik)
LSTM = *Long Short-Term Memory* (uzoq-qisqa muddatli xotira)

GRU — ixchamroq arxitektura: 2 ta darvoza ishlatadi, alohida xotira katagi (*cell state*) yo'q.

Amaliy mazmuni: kamroq parametr sababli ko'pincha tezroq o'qitiladi va kichikroq ma'lumotda qulay.

LSTM — nazorati kuchliroq arxitektura: 3 ta darvoza va alohida xotira katagi (*cell state*) mavjud.

Amaliy mazmuni: uzoq bog'liqliklarni saqlashda ko'pincha barqarorroq ishlaydi.

Bog'lovchi g'oya: ikkalasi ham 7-ma'ruzadagi *vanishing gradient* (yo'qoluvchi gradient) muammosini **darvozalar** orqali yumshatadi, lekin uni mutlaq yo'q qilib yubormaydi.

Prezentatsiya rejasi

- 1 Kirish: uzoq xotirani darvozalar bilan saqlaymiz
- 2 Yo'qoluvchi gradient (Vanishing gradient) va darvoza g'oyasi
- 3 GRU: axborot oqimini darvozalar boshqaradi
- 4 LSTM: cell state va uch darvoza
- 5 GRU va LSTM ni taqqoslash
- 6 Xulosa, manbalar va keyingi qadam

Takrorlash: o'tgan darsdan ikki savol

1-savol

7-ma'ruzada oddiy RNN gradienti uzoq qadamlarda **so'nishi** aytilgan edi. Buning asosiy sababi nima?

Maslahat: tanh hosilasi va Jakobianlar ko'paytmasi.

2-savol

7-amaliyotda RNN klassifikatori qisqa sharhlarda ishladi. Lekin uzun gaplarda, ayniqsa muhim so'z uzoqda kelsa, nega model qiynaladi?

Takrorlash: o'tgan darsdan ikki savol

1-savol

7-ma'ruzada oddiy RNN gradienti uzoq qadamlarda **so'nishi** aytilgan edi. Buning asosiy sababi nima?

Maslahat: tanh hosilasi va Jakobianlar ko'paytmasi.

1-javob

$$\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(1 - h_t^{\odot 2}) W_h.$$

Gradient orqaga qaytganda bunday Jakobianlar ko'payadi. Agar ularning normasi ko'pincha < 1 bo'lsa, ko'paytma eksponensial so'nadi.

2-savol

7-amaliyotda RNN klassifikatori qisqa sharhlarda ishladi. Lekin uzun gaplarda, ayniqsa muhim so'z uzoqda kelsa, nega model qiynaladi?

2-javob

Uzoq bog'liqlik bo'yicha gradient zaiflashadi. Shu sabab model gap boshidagi yoki uzoqdagi muhim signalni barqaror saqlay olmaydi.

Bugun bu muammoni **darvozalar** (*gates*) yordamida yumshatamiz.

Bugungi dars maqsadlari

- 1 Oddiy RNN nega uzoq bog'liqlikni barqaror saqlay olmasligini *vanishing gradient* (yo'qoluvchi gradient) orqali **tushuntira olasiz**.
- 2 GRU (*Gated Recurrent Unit*) dagi **yangilash darvozasi** (*update gate*) va **qayta sozlash darvozasi** (*reset gate*) axborot oqimini qanday boshqarishini **izohlay olasiz**.
- 3 LSTM dagi **xotira katagi** (*cell state*) nega gradient uchun additiv “magistral” vazifasini bajarishini **tushuntira olasiz**.
- 4 LSTM **unutish darvozasi** (*forget gate*) qiymatini qo'lda hisoblab, masalan $f_t = \sigma(1.5) \approx 0.818$, GRU va LSTM ni **taqqoslay olasiz**.

Oldingi bilimlar: RNN rekurrensiyasi, yo'qoluvchi gradient, sigmoid σ , tanh, element bo'yicha ko'paytma ($\odot$) va zanjir qoidasi.

Bosqich	Mavzu	Natija
1	Yo'qoluvchi gradient(anishing gradient) va darvoza g'oyasi	0.9^{10} vs 0.4^{10}
2	GRU: yangilash / reset darvozalari	$z = \sigma(1) \approx 0.731$
3	LSTM: cell state + uch darvoza	$f_t = \sigma(1.5) \approx 0.818$
4	GRU va LSTM ni taqqoslash	parametr soni
–	Xulosa va 8-amaliyotga ko'prik	amaliyot yo'nalishi

Darsning asosiy zanjiri: oddiy RNN uzoq xotirada qiynaladi → darvozalar axborot oqimini boshqaradi → GRU ixcham yechim beradi → LSTM alohida *cell state* orqali uzoq bog'liqlikni kuchliroq saqlaydi.

Bugungi $f_t = \sigma(1.5) \approx 0.818$ hisoblash namunasi 8-amaliyotda kod orqali tekshiriladi.

Bugungi muammo: kerakli xotirani saqlab qolish

Oddiy RNN qayerda qiynaladi?

Holat har qadamda qayta hisoblanadi

$$h_t = \tanh(W_h h_{t-1} + W_x x_t)$$

Har qadamda eski holat yangi kirish bilan qayta transformatsiya qilinadi. $\tanh$ qiymatlarni siqadi, shuning uchun uzoqdagi signal asta-sekin **zaiflashishi** mumkin.

Faqat o'lchamni oshirish yetarli emas

Hidden state o'lchamini oshirish model sig'imini ko'paytiradi, lekin gradient hali ham vaqt bo'ylab **ko'paytmali yo'l** orqali qaytadi.

Bugungi yechim g'oyasi

- 1 **Darvozalar** (*gates*): qaysi axborotni saqlash va qaysisini yangilashni modelning o'zi o'rganadi.
- 2 **GRU**: eski holat va yangi nomzod holat orasini darvoza orqali aralashtiradi.
- 3 **LSTM cell state**: xotirani additiv yo'l bilan olib yuradi, ya'ni gradient uchun "magistral" yaratadi.

Asosiy fikr: GRU va LSTM oddiy RNN dagi yo'qoluvchi gradient muammosini darvozali xotira orqali **yumshatadi**; lekin bu muammoni mutlaq yo'q qilib yubormaydi.

Prezentatsiya rejasi

- 1 Kirish: uzoq xotirani darvozalar bilan saqlaymiz
- 2 Yo'qoluvchi gradient (Vanishing gradient) va darvoza g'oyasi
- 3 GRU: axborot oqimini darvozalar boshqaradi
- 4 LSTM: cell state va uch darvoza
- 5 GRU va LSTM ni taqqoslash
- 6 Xulosa, manbalar va keyingi qadam

Intuitsiya: signal ko'p qadamda kichrayib boradi

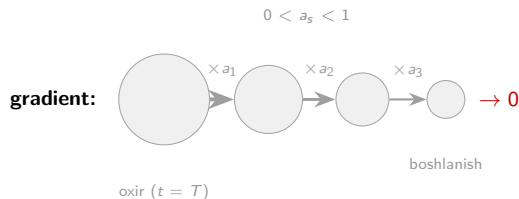
RNN da gradient oxirgi qadamdan boshlanish tomonga **zanjir qoidasi** orqali qaytadi. Har qadamda gradient lokal hosila/Jakobianga ko'payadi.

Agar bu lokal koeffitsientlar ko'pincha 1 dan kichik bo'lsa, ko'paytma tez kichrayadi:

$$0 < a < 1 \quad \implies \quad a^n \xrightarrow{n \rightarrow \infty} 0.$$

Shuning uchun uzoqdagi signal, masalan gap boshidagi muhim so'z, o'qitish vaqtida oxirgi qarorga juda zaif ta'sir qilishi mumkin.

Eslatma: muammo faqat gap uzunligida emas; asosiy sabab — uzun zanjir bo'ylab *ko'paytmali gradient* yo'li.



Xulosa: har qadamda kichik koeffitsientlarga ko'payish gradientni eksponensial kamaytiradi. Bu **vanishing gradient** (yo'qoluvchi gradient) muammosi.

Qo'lda misol: 0.9^{10} qoladi, 0.4^{10} deyarli yo'qoladi

Bir qadamdagi gradient koeffitsienti a bo'lsa, n qadamdan keyin gradient ulushi taxminan a^n bo'ladi.

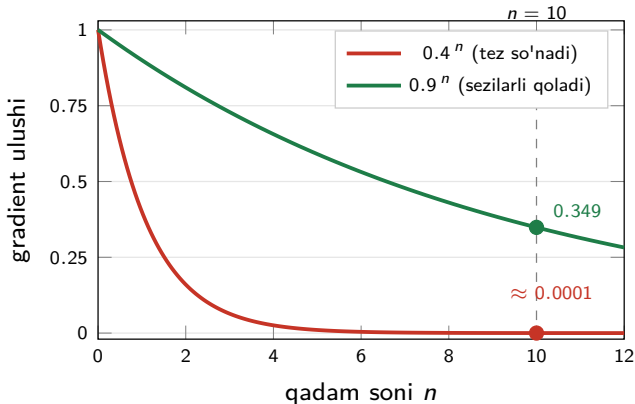
$0.9^{10} \approx \mathbf{0.349}$ $\Rightarrow$ taxminan 35% qoladi,

$0.4^{10} \approx \mathbf{0.000105}$ $\Rightarrow$ deyarli 0.

Muhim farq: 0.9 va 0.4 orasidagi farq atigi 0.5, lekin 10 qadamdan keyin natijalar

$$\frac{0.9^{10}}{0.4^{10}} \approx 3325$$

marta farq qiladi. Bu — **eksponensial ta'sir**.



Xulosa: gradient har qadamda kichik koeffitsientlarga ko'payaversa, uzoqdagi signal o'qitishga juda zaif ta'sir qiladi.

Keltirib chiqarish: muammoning ildizi nimada?

1-qadam. Uzoqdagi h_t ning h_T ga ta'siri zanjir qoidasi orqali Jakobianlar ko'paytmasiga aylanadi:

$$\frac{\partial h_T}{\partial h_t} = \prod_{s=t+1}^T J_s, \quad J_s = \frac{\partial h_s}{\partial h_{s-1}} = \text{diag}(1 - h_s^{\odot 2}) W_h.$$

2-qadam. Demak, gradient bitta hosilaga emas, ko'p qadamli **ko'paytmali yo'l**ga bog'liq:

$$\left\| \frac{\partial h_T}{\partial h_t} \right\| \leq \prod_{s=t+1}^T \|J_s\|.$$

Agar ko'p bosqichlarda $\|J_s\| < 1$ bo'lsa, ko'paytma eksponensial so'nadi (*vanishing gradient*); agar $\|J_s\| > 1$ bo'lsa, gradient portlashi mumkin (*exploding gradient*).

3-qadam. Muammo faqat ketma-ketlik uzunligida emas; asosiy sabab — har qadamda holat W_h orqali qayta transformatsiya qilinishi va gradientning shu uzun ko'paytmali yo'ldan qaytishidir.

Xulosa: bizga holatni har qadam to'liq qayta yozmaydigan, balki *nimani saqlash* va *nimani yangilashni* tanlaydigan mexanizm kerak. Bu mexanizm — **darvoza** (*gate*).

Yangi g'oya: o'rganiladigan “darvoza”

Darvoza (*gate*) — 0 va 1 orasidagi o'rganiladigan koeffitsient:

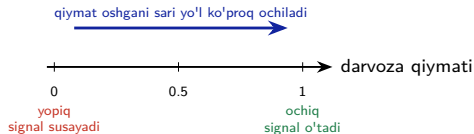
“shu yo'ldan axborotning qanchasi o'tsin?”

- Darvoza $\approx 1 \Rightarrow$ shu yo'l **ochiq**: signal kuchli o'tadi.
- Darvoza $\approx 0 \Rightarrow$ shu yo'l **yopiq**: signal bostiriladi.

Aslida darvoza bitta son emas, ko'pincha **vektor**: hidden state ning har bir elementi uchun alohida 0–1 koeffitsient.

Model bu koeffitsientlarni *ma'lumotdan o'rganadi*: qaysi signalni o'tkazish, qaysisini susaytirishni o'zi tanlaydi.

Eslatma: darvozaning aniq ma'nosi u qaysi yo'lga qo'llanishiga bog'liq. Masalan, LSTM forget gate va GRU update gate bir xil talqin qilinmaydi.



Intuitsiya: darvoza — “yarim ochiq eshik”. 0 ga yaqin qiymat signalni susaytiradi, 1 ga yaqin qiymat esa signalni o'tkazadi.

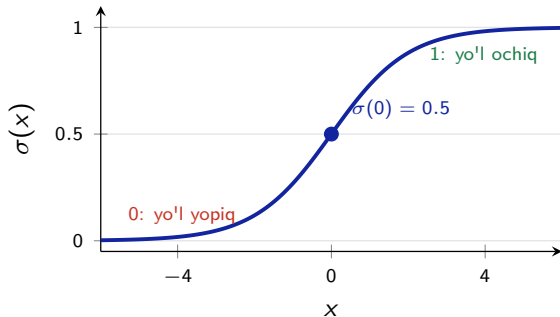
Eslatma: sigmoid σ — darvoza uchun tabiiy tanlov

$$\sigma(x) = \frac{1}{1 + e^{-x}} \in (0, 1)$$

Sigmoid har qanday haqiqiy sonni $(0, 1)$ oraliqqa **siqadi**. Shuning uchun u darvozaning “ochiqlik darajasi”ni ifodalash uchun qulay.

$$\sigma(0) = 0.5$$

- katta musbat x bo'lsa: $\sigma(x) \approx 1$ (yo'l ochiq);
- katta manfiy x bo'lsa: $\sigma(x) \approx 0$ (yo'l yopiq).



Ko'prik: standart GRU darvozalari (z, r) va LSTM darvozalari (f, i, o) odatda sigmoid orqali hisoblanadi. Shu sabab ularning qiymati 0 va 1 orasida bo'ladi.

Prezentatsiya rejasi

- 1 Kirish: uzoq xotirani darvozalar bilan saqlaymiz
- 2 Yo'qoluvchi gradient (Vanishing gradient) va darvoza g'oyasi
- 3 GRU: axborot oqimini darvozalar boshqaradi**
- 4 LSTM: cell state va uch darvoza
- 5 GRU va LSTM ni taqqoslash
- 6 Xulosa, manbalar va keyingi qadam

Intuitsiya: eski va yangi holat orasida “aralashtirish”

GRU har qadamda eski xotira va yangi nomzod holatni **qancha aralashtirishni** darvoza orqali boshqaradi.

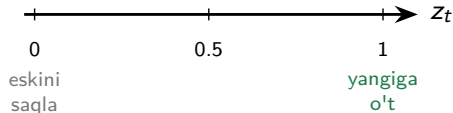
- **Yangilash darvozasi** (*update gate*) z_t : eski holat h_{t-1} saqlansinmi yoki yangi nomzod $\tilde{h}_t$ kuchliroq kirsinmi?
- **Qayta sozlash darvozasi** (*reset gate*) r_t : yangi nomzod holatni hisoblashda eski kontekstning qanchasi ishlatilsin?

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

$z_t \approx 0$ bo'lsa, h_t eski holatga yaqin; $z_t \approx 1$ bo'lsa, h_t yangi nomzodga yaqin.

Oddiy RNN har qadamda holatni deyarli to'liq qayta hisoblaydi. **GRU** esa holatni *tanlab yangilaydi*. Shu sabab uzoqdagi signalni saqlab qolish imkoniyati ortadi.

z_t ortgani sari h_t yangi nomzodga yaqinlashadi



Pedagogik obraz: z_t — aralashtirgich dastagi.

GRU darvozalari: yangilash z_t va qayta sozlash r_t

$$z_t = \sigma(W_z x_t + U_z h_{t-1})$$

z_t eski holat h_{t-1} va yangi nomzod $\tilde{h}_t$ orasidagi **aralashtirish ulushi**.

- $z_t \approx 0 \Rightarrow$ eski holat ko'proq saqlanadi;
- $z_t \approx 1 \Rightarrow$ yangi nomzod holat kuchliroq kiradi.

Keyingi slaydda bu quyidagi aralashmada ishlatiladi:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t.$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1})$$

r_t yangi nomzod $\tilde{h}_t$ hisobida eski holatning **qanchasi ishlatilishini** boshqaradi.

- $r_t \approx 0 \Rightarrow$ eski kontekst ta'siri kamayadi;
- $r_t \approx 1 \Rightarrow$ eski kontekst nomzodga kuchliroq kiradi.

Nomzod holat odatda shunday quriladi:

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1})).$$

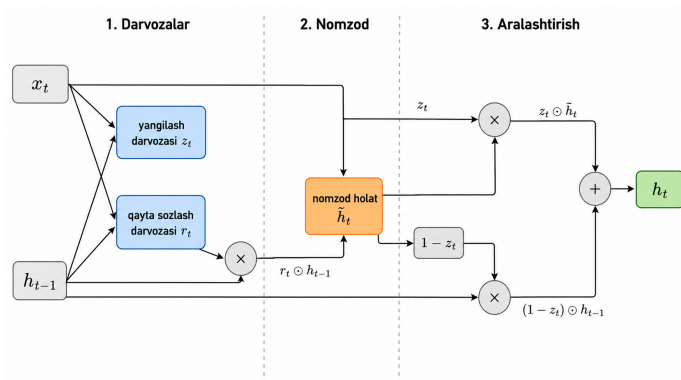
Belgilar: x_t — joriy kirish, h_{t-1} — oldingi hidden state, σ — sigmoid, $\odot$ — element bo'yicha ko'paytma. W_z, U_z, W_r, U_r — o'rganiladigan og'irliklar. Bu yerda bias hadlari soddalik uchun ko'rsatilmagan.

GRU: to'liq rasmiy ta'rif

$$z_t = \sigma(W_z x_t + U_z h_{t-1}), r_t = \sigma(W_r x_t + U_r h_{t-1}),$$
$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1})),$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t.$$

Belgilar: z_t — yangilash darvozasi; r_t — qayta sozlash darvozasi; $\tilde{h}_t$ — nomzod holat; $\odot$ — element bo'yicha ko'paytma.

Bias hadlari soddalik uchun ko'rsatilmagan.



Asosiy g'oya: r_t nomzod holatni hisoblashda eski kontekst ta'sirini sozlaydi. z_t esa eski holat h_{t-1} va yangi nomzod $\tilde{h}_t$ ni aralashtirib, yakuniy h_t ni beradi.

Keltirib chiqarish: GRU yangilanishi qayerdan keladi?

1-qadam. Oddiy RNN da

$$h_t = \tanh(\dots)$$

har qadamda holat yangi qiymatga qayta hisoblanadi.

2-qadam. G'oya: eski holat va yangi nomzod orasida *interpolatsiya* qilamiz:

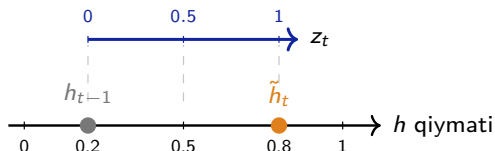
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

3-qadam. z_t ni o'rganiladigan darvoza qilamiz:

$$z_t = \sigma(\cdot) \in (0, 1)$$

$z_t \rightarrow 0$ bo'lsa eski holat ko'proq saqlanadi, $z_t \rightarrow 1$ bo'lsa yangi nomzodga o'tiladi.

Xulosa: model qachon yangilashni o'zi o'rganadi. Uzoq xotira kerak bo'lsa, z_t kichikroq qiymat tanlashi mumkin.



z_t — “aralashtirgich dastagi”: h_t ni h_{t-1} dan $\tilde{h}_t$ tomonga suradi.

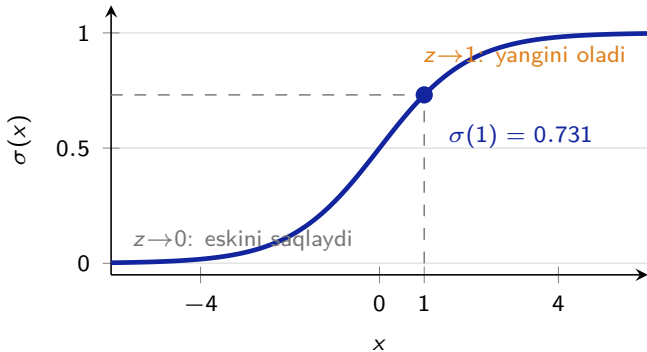
Qo'lda misol: GRU yangilash darvozasi (*update gate*) — $z = \sigma(1) = 0.731$

$h_{t-1} = 0.2$, $\tilde{h}_t = 0.8$, pre-aktivatsiya = 1.0 (bir o'lchamli, soddalashtirilgan):

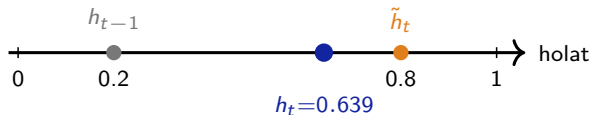
$$z_t = \sigma(1.0) = \frac{1}{1 + e^{-1}} = \mathbf{0.731}$$

$$\begin{aligned} h_t &= (1 - 0.731) \cdot 0.2 + 0.731 \cdot 0.8 \\ &= 0.054 + 0.585 = \mathbf{0.639} \end{aligned}$$

$z = 0.731 \Rightarrow$ natija asosan yangi nomzodga yaqin (“~ 73% yangi, ~ 27% eski”). z kichik bo'lsa, eski holat ko'proq saqlanardi.



z qancha katta bo'lsa, natija yangiga shuncha yaqin:



Sizning vazifangiz: yarim ochiq yangilash darvozasi

Vazifa

Agar $z_t = \sigma(0) = 0.5$ bo'lsa ($h_{t-1} = 0.2$, $\tilde{h}_t = 0.8$):

$$h_t = ?$$

Sizning vazifangiz: yarim ochiq yangilash darvozasi

Vazifa

$$z_t = \sigma(0) = 0.5, h_{t-1} = 0.2, \tilde{h}_t = 0.8: h_t = ?$$

Javob

$$h_t = (1 - 0.5) \cdot 0.2 + 0.5 \cdot 0.8 = 0.1 + 0.4 = 0.5$$

$z = 0.5$ — eski va yangi teng aralashdi. $z \rightarrow 0$ bo'lsa $h_t \rightarrow h_{t-1}$ (xotira to'liq saqlanadi).

Kod va formula: GRU katagi

Python kodi

```
import numpy as np

def sigmoid(x):
    return 1/(1+np.exp(-x))

def gru_step(h_prev, x, Wz,Uz, Wr,Ur, Wh,Uh):
    z = sigmoid(Wz*x + Uz*h_prev)          # update
    r = sigmoid(Wr*x + Ur*h_prev)          # reset
    h_tilde = np.tanh(Wh*x + Uh*(r*h_prev)) # candidate
    h = (1-z)*h_prev + z*h_tilde           # mix
    return h, z, r, h_tilde

h, z, r, ht = gru_step(0.2, 1.0, 1.0, 0.0, 1.1)
print(round(h,3))    # 0.639
```

Kod va formula mosligi

Kod	Formula
z	z_t
r	r_t
h_tilde	$\tilde{h}_t$
h	h_t

Asosiy tuzatish: r endi nomzod holatni hisoblashda ishlaydi:

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1})).$$

Amaliy kutubxonalarda `nn.GRU` shu hisobni optimallashtirilgan ko'rinishda bajaradi.

GRU afzalligi va tuzog'i: darvoza — sehr emas

Afzalliklari

- Kam parametr (LSTM ning $\sim 3/4$ i) $\Rightarrow$ tezroq.
- Kichik ma'lumotda barqaror o'rganadi.
- Vanishing gradientni **yumshatadi**.

Cheklovlar

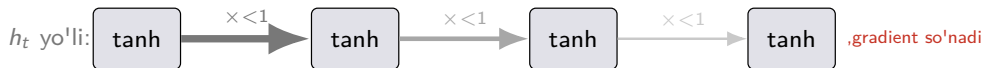
- Vanishing gradientni butunlay yo'qotmaydi.
- Juda uzoq bog'liqlikda hali ham qiynalishi mumkin.

Yechim yo'nalishi: alohida **cell state** bilan ma'lumotni additiv olib yurish — LSTM (keyingi bo'lim).

Prezentatsiya rejasi

- 1 Kirish: uzoq xotirani darvozalar bilan saqlaymiz
- 2 Yo'qoluvchi gradient (Vanishing gradient) va darvoza g'oyasi
- 3 GRU: axborot oqimini darvozalar boshqaradi
- 4 LSTM: cell state va uch darvoza**
- 5 GRU va LSTM ni taqqoslash
- 6 Xulosa, manbalar va keyingi qadam

Intuitsiya: ma'lumot uchun “magistral yo'l”



h_t yo'lida gradient har qadamda $\tanh$ va og'irliklar orqali o'tadi; c_t yo'lida esa xotira additiv “magistral” orqali uzatiladi. Agar $f_t \approx 1$ bo'lsa, uzoq signal yaxshiroq saqlanadi.

Ikki holat: cell state c_t va hidden state h_t

Cell state c_t — uzoq xotira

- Ma'lumot “magistrali” — vaqt bo'ylab **additiv** oqadi.
- Tashqariga to'g'ridan chiqmaydi — *ichki* xotira chizig'i.
- Uzoq bog'liqliklarda gradient oqimini saqlashga yordam beradi.

- Har qadamdagi **tashqi chiqish** (keyingi qatlamga yoki bashoratga).
- c_t dan *output darvoza* (o_t) orqali shakllanadi: $[h_t = o_t \odot \tanh(c_t).]$
- Oddiy RNN dagi h_t vazifasiga yaqin rol bajaradi.

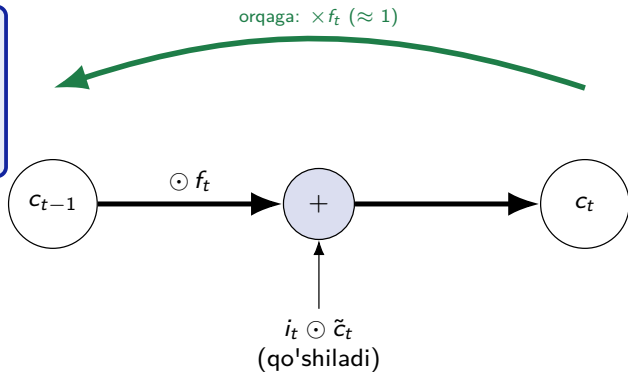
Asosiy farq: oddiy RNN da bitta holat h_t ham xotira, ham chiqish edi. LSTM bularni ajratadi: c_t uzoq xotirani saqlaydi, h_t esa shu xotiraning nazoratlangan chiqishidir.

Nega cell state gradientni saqlaydi: additiv yo'l

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t \quad (\text{asosiy yo'l})$$

- c_{t-1} ga **matritsa ko'paytma yo'q** — faqat f_t ga.
- Uzoq masofa: $\frac{\partial L}{\partial c} \approx \prod_t f_t$. $f_t \approx 1 \Rightarrow$ saqlanadi.
- RNN: $\prod \text{diag}(1 - h^2) W_h$ — siqiladi yoki portlaydi.



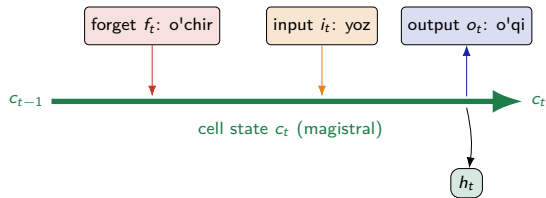
Oldinga: $\times f_t$, keyin $+$. Orqaga ham faqat $\times f_t$ — tanh siqilishi yo'q.

Intuitsiya: uchta darvoza — o'chir, yoz, o'qi

LSTM cell state ni **uch** darvoza boshqaradi:

- **forget** f_t (*unutish*): eski xotiradan nimani *o'chirish*.
- **input** i_t (*kiritish*): yangi ma'lumotdan nimani *yozish*.
- **output** o_t (*chiqarish*): xotiradan nimani *o'qib* chiqarish (h_t ga).

Har biri sigmoid orqali hisoblanadi, shuning uchun $f_t, i_t, o_t \in (0, 1)$ — “yarim ochiq eshik”.



Darvozalar — o'rganiladigan filtrlar; qaysi ma'lumot muhimligini model ma'lumotdan o'rganadi.

Darvozama-darvoza: forget, input, output

Forget f_t

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Input i_t

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Eski xotira c_{t-1} ning qanchasi **qoladi**.
 $f_t \approx 1$ — saqla; $f_t \approx 0$ — o'chir.

Yangi nomzod $\tilde{c}_t$ ning qanchasi **yoziyadi**.
 $\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$.

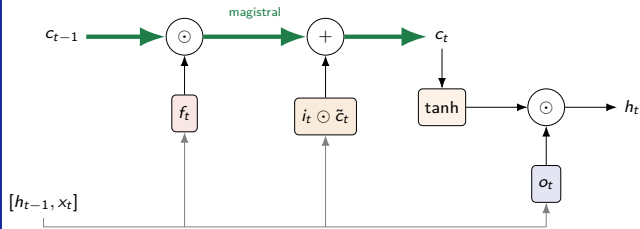
Xotiradan h_t ga **nima chiqadi**:
 $h_t = o_t \odot \tanh(c_t)$.

$[h_{t-1}, x_t]$ — birlashtirilgan vektor (*concat*). Uchchala darvoza bir xil ko'rinishda — faqat o'z og'irliklari va vazifasi farqli. Nomzod $\tilde{c}_t$ — tanh orqali “yangi ma'lumot” (*candidate memory*).

LSTM: to'liq rasmiy ta'rif

$$\begin{aligned}f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) \\i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \\o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \\\tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c) \\c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\h_t &= o_t \odot \tanh(c_t)\end{aligned}$$

$[h_{t-1}, x_t]$ — birlashtirilgan vektor; c_t — cell state (uzoq xotira); h_t — yashirin holat (chiqish).



c_t magistral; f, i, o darvozalari uni boshqaradi; chiqish
 $h_t = o_t \odot \tanh(c_t)$.

Qo'lda hisob: forget gate — $f_t = \sigma(1.5) = 0.818$

$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$ (bir o'lchamli, soddalashtirilgan), $h_{t-1} = 0.5$, $x_t = 1.0$, $W_f = [1, 1]$, $b_f = 0$:

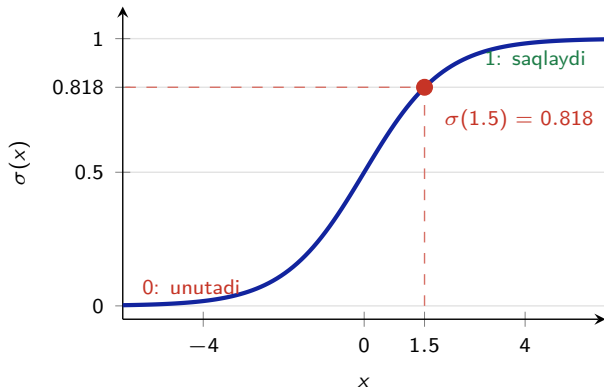
$$f_t = \sigma(1 \cdot 0.5 + 1 \cdot 1.0 + 0) = \sigma(1.5)$$

$$= \frac{1}{1 + e^{-1.5}} \approx \mathbf{0.818}$$

$f_t = 0.818$ (1 ga yaqin) $\Rightarrow$ o'tmish xotirasining $\sim 81.8\%$ saqlanadi.

Ertangi 8-amaliyot birinchi assert:

$$\text{abs}(f_t - 0.818) < 1e-3$$



Sizning vazifangiz: cell state ni yangilang

Vazifa

$f_t = 0.818$, $c_{t-1} = 1.0$, input gate $i_t = 0.5$, nomzod $\tilde{c}_t = 0.6$:

$$c_t = f_t c_{t-1} + i_t \tilde{c}_t = ?$$

Sizning vazifangiz: cell state ni yangilang

Vazifa

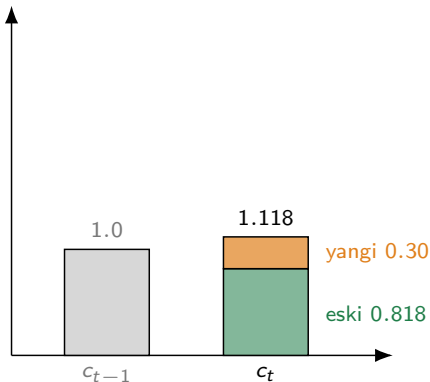
$$f_t = 0.818, c_{t-1} = 1.0, i_t = 0.5, \tilde{c}_t = 0.6: c_t = ?$$

Javob

$$c_t = 0.818 \cdot 1.0 + 0.5 \cdot 0.6 = 0.818 + 0.30 = 1.118$$

Eski xotiradan 0.818 saqlandi, yangidan 0.30 qo'shildi — **additiv** yangilanish (siqilish yo'q).

c qiymati



$$f_t \odot c_{t-1} \text{ (saqlangan)} + i_t \odot \tilde{c}_t \text{ (qo'shilgan)}.$$

Kod va formula: LSTM katagi

Python kodi

```
import numpy as np
def sigmoid(x): return 1/(1+np.exp(-x))
def lstm_step(h, c, x, Wf,Wi,Wo,Wc):
    z = np.array([h, x])
    f = sigmoid(Wf @ z)    # forget
    i = sigmoid(Wi @ z)    # input
    o = sigmoid(Wo @ z)    # output
    c_tilde = np.tanh(Wc @ z)
    c = f*c + i*c_tilde    # cell state
    h = o*np.tanh(c)
    return h, c
# misol: f = sigmoid(0.5+1.0) = sigmoid(1.5) = 0.818
```

Kod va formula mosligi

Kod	Formula
f	f_t (forget)
i	i_t (input)
o	o_t (output)
$f*c+i*c_tilde$	c_t
$o*\tanh(c)$	h_t

Amalda nn.LSTM ishlatiladi; 8-amaliyotda (m08) GRU bilan taqqoslanadi.

LSTM tuzog'i: kuch — narx bilan keladi

Hisoblash narxi

- 4 ta darvoza/nomzod $\Rightarrow$ oddiy RNN dan $\sim 4\times$ ko'p parametr.
- Kichik ma'lumotda ortiqcha moslashish (*overfitting*) xavfi.
- Sekinroq o'qiydi (ko'proq hisob).
- Vanishing gradientni faqat **yumshatadi** (butunlay yo'qotmaydi); exploding gradient hamon mumkin — *gradient clipping* kerak bo'lishi mumkin.

Yechim: kam ma'lumotda GRU (kamroq parametr) ko'pincha yetarli — keyingi bo'limda taqqoslaymiz.

Amaliy maslahat: forget darvozasi biasini musbat ($b_f = +1$) boshlash boshida xotirani saqlashga moyil qiladi (ilovaga qarang).

Prezentatsiya rejasi

- 1 Kirish: uzoq xotirani darvozalar bilan saqlaymiz
- 2 Yo'qoluvchi gradient (Vanishing gradient) va darvoza g'oyasi
- 3 GRU: axborot oqimini darvozalar boshqaradi
- 4 LSTM: cell state va uch darvoza
- 5 GRU va LSTM ni taqqoslash**
- 6 Xulosa, manbalar va keyingi qadam

Intuitsiya: bir xil g'oya, ikki “vazn toifasi”

Ikkalasi ham darvozalar bilan uzoq xotirani saqlaydi:

- **LSTM**: alohida cell state, 3 darvoza — murakkabroq, og'irroq, nazoratliroq.
- **GRU**: cell state yo'q (h birlashgan), 2 darvoza — yengilroq, tezroq.

Ko'p vazifada natijalari yaqin bo'lishi mumkin — tanlov ma'lumot, ketma-ketlik uzunligi va resursga bog'liq.

Amaliy qoida: kam ma'lumot / tez kerak $\Rightarrow$ GRU; uzoq bog'liqlik va yetarli ma'lumot $\Rightarrow$ LSTM dan boshlang.

GRU va LSTM: yonma-yon taqqoslash

Xususiyat	GRU	LSTM
Darvozalar soni	2 (z, r)	3 (f, i, o)
Alohida cell state	yo'q	ha (c_t)
Parametrlar	kamroq ($\approx 3/4$)	ko'proq
Tezlik	tezroq	sekinroq
Uzoq bog'liqlik barqarorligi	yaxshi	yaxshi (biroz nazoratliroq)
Qachon ishlatish	kam ma'lumot / tezlik	uzoq ketma-ketlik / ko'p ma'lumot

Universal g'olib yo'q. Natija ma'lumot hajmi, ketma-ketlik uzunligi va hisoblash byudjetiga bog'liq. To'g'ri yondashuv — ikkalasini bir xil vazifada sinab, F1 va tezlikni o'lchash.

Parametr soni: GRU $\approx 75\%$ LSTM

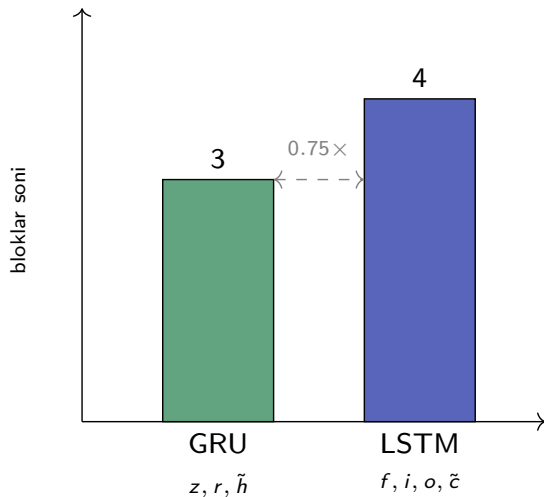
Har darvoza/nomzod bitta og'irlik bloki: $dH + H^2 + 2H$.

$$\text{LSTM} \approx 4(dH + H^2 + 2H) \quad (f, i, o, \tilde{c})$$

$$\text{GRU} \approx 3(dH + H^2 + 2H) \quad (z, r, \tilde{h})$$

$$\frac{\text{GRU}}{\text{LSTM}} = \frac{3}{4} = \mathbf{0.75}$$

d — kirish o'lchami; H — yashirin o'lcham. $2H$ — PyTorch ikki bias saqlaydi (b_{ih}, b_{hh}). GRU forget+input ni bitta z ga birlashtiradi $\Rightarrow$ 3 blok; $\sim 25\%$ kam parametr $\Rightarrow$ tezroq, kichik ma'lumotda barqarorroq. (Nisbat faqat blok soniga bog'liq.)



Qo'lda misol: parametr sonini sanaymiz

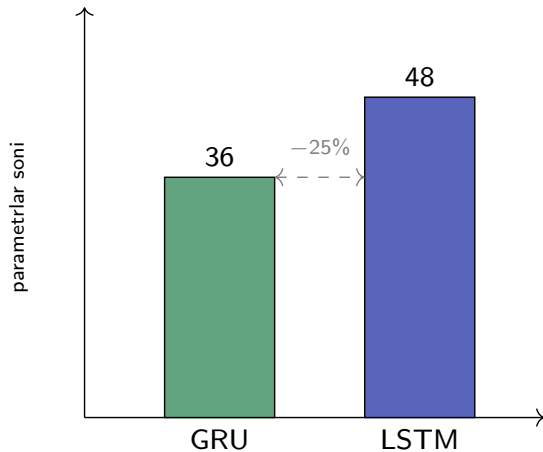
$d = 2$, $H = 2$ uchun bitta blok:

$$dH + H^2 + 2H = 2 \cdot 2 + 2 \cdot 2 + 2 \cdot 2 = 12$$

$$\text{LSTM} = 4 \times 12 = \mathbf{48}$$

$$\text{GRU} = 3 \times 12 = \mathbf{36}$$

GRU 36 vs LSTM 48 — 25% kam. $2H$ — PyTorch ikki bias (b_{ih} , b_{hh}); aynan shu sonni `model.parameters()` da ko'rasiz. Katta H da farq absolyut sonda o'sadi, nisbat 0.75 saqlanadi.



Sizning vazifangiz: kattaroq model uchun parametr

Vazifa

$d = 3$, $H = 2$ uchun bitta blok $= 3 \cdot 2 + 2 \cdot 2 + 2 \cdot 2 = 14$. LSTM va GRU parametrlari soni nechta?

Sizning vazifangiz: kattaroq model uchun parametr

Vazifa

Bitta blok = 14. LSTM va GRU parametrlari soni?

Javob

$$\text{LSTM} = 4 \times 14 = 56; \quad \text{GRU} = 3 \times 14 = 42.$$

Nisbat hamon $48/56 = 0.75$ — darvoza bloklari soni farqi (3 vs 4) saqlanadi.

Kod va formula: PyTorch nn.LSTM / nn.GRU

Python kodi

```
import torch.nn as nn

# 100 = input_size, 128 = hidden_size
# Interfeys bir xil; faqat klass nomi farq qiladi
lstm = nn.LSTM(100, 128, batch_first=True)
gru = nn.GRU(100, 128, batch_first=True)

def n_params(m):
    return sum(p.numel() for p in m.parameters())

print(n_params(lstm)) # 117760 = 4 * 29440
print(n_params(gru)) # 88320 = 3 * 29440 (0.75x)
```

Kod va formula mosligi

Kod	Formula
nn.LSTM	4 blok
nn.GRU	3 blok
1 blok	$dH + H^2 + 2H$
nisbat	$3/4 = 0.75$

8-amaliyotda m08 aynan shu ikki qatlamni bir xil vazifada taqqoslaydi (compare_report).

Prezentatsiya rejasi

- 1 Kirish: uzoq xotirani darvozalar bilan saqlaymiz
- 2 Yo'qoluvchi gradient (Vanishing gradient) va darvoza g'oyasi
- 3 GRU: axborot oqimini darvozalar boshqaradi
- 4 LSTM: cell state va uch darvoza
- 5 GRU va LSTM ni taqqoslash
- 6 Xulosa, manbalar va keyingi qadam

O'zbek tilida: nega uzoq xotira muhim

Bitta o'zak ustiga ko'p qo'shimcha tizilib ketadi:

o'rgan-il-gan-lar-dan

Ma'no oxirgi qo'shimchagacha cho'ziladi — model boshidagi o'zakni oxirigacha “eslab” turishi kerak.

SOV tartibi: kesim (fe'l) gap *oxirida* keladi. Demak bosh bo'lakdagi ega bilan oxirdagi kesim orasidagi bog'liqlik uzoq masofali.



Kichik korpusda GRU (kamroq parametr) ko'pincha LSTM dan *barqarorroq* o'rganadi. Hajmga qarab tanlang.

Sintez: uchta arxitektura bir qarashda (RNN $\rightarrow$ GRU $\rightarrow$ LSTM)

Model	Darvoza	Parametr	Vanishing'ga qarshilik
Oddiy RNN	0	eng kam	zaif ($\prod (1 - h^2) W_h \rightarrow 0$)
GRU	2 (z, r)	$\approx 0.75 \times$	yaxshi (interpolatsion)
LSTM	3 (f, i, o)	eng ko'p	eng yaxshi (additiv cell state)

Darvozalar axborot oqimini boshqaradi. **Additiv** cell state (LSTM) yoki **interpolatsion** yangilanish (GRU) tufayli orqaga tarqalishda har bir qadam omili $\partial c_t / \partial c_{t-1} = f_t \approx 1$ bo'lib, ularning ko'paytmasi $\prod_t f_t$ tez nolga intilmaydi — vanishing gradient sezilarli *kamayadi* (butunlay yo'q bo'lmaydi; exploding uchun baribir clipping kerak).

Asosiy manbalar: LSTM va GRU qayerdan kelgan

Hochreiter & Schmidhuber (1997)

“Long Short-Term Memory”

Neural Computation **9**(8):1735–1780.

Cho va boshqalar (2014)

“Learning Phrase Representations. . .” (GRU)

EMNLP.

Nega muhim:

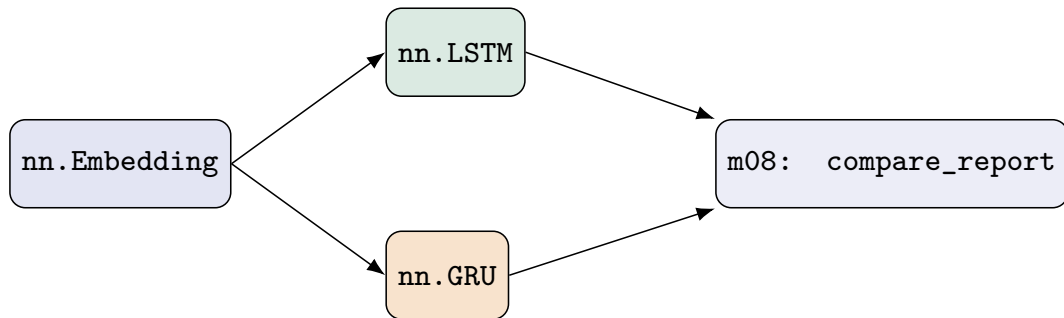
- LSTM — vanishing gradient muammosiga muhim yechim bergan birinchi ish.
- “Constant error carousel” = bizning additiv cell state.
- GRU (2014) — LSTM ni ixchamlashtirdi; tarjima uchun yaratilgan.
- 25+ yildan keyin ham asosiy g'oya o'zgarmagan.

Bugungi maqsadlar bajarildi

- ✓ Vanishing gradient: $0.9^{10} \approx 0.349$ saqlanadi, $0.4^{10} \approx 0.0001$ so'nadi.
- ✓ yangilash darvozasi: $z = \sigma(1) = 0.731$ — yangi va eski xotirani aralashtiradi.
- ✓ Additiv cell state: $\frac{\partial c_t}{\partial c_{t-1}} = f_t$ — tanh siqilishi yo'lq.
- ✓ Forget darvozasi: $f_t = \sigma(1.5) = 0.818$ — xotiraning $\sim 82\%$ qoladi.
- ✓ Parametr narxi: GRU $\approx 0.75 \times$ LSTM — 30 va 40 (bizning misol).

Bir jumlada: darvoza qiymati $\sigma(\cdot) \in (0, 1)$ axborotning qancha qismi o'tishini boshqaradi; additiv yangilash gradient oqimini yaxshiroq saqlaydi.

Ertaga — 8-amaliyot: GRU va LSTM ni amalda taqqoslash



- `m08`: bitta `GRULSTMClassifier`, `arch='lstm'` yoki `'gru'`.
- Bir xil vazifada F1 va tezlikni o'lchaymiz.
- Forget darvozasi qiymatini tekshiramiz.

Tekshiruv (assert)

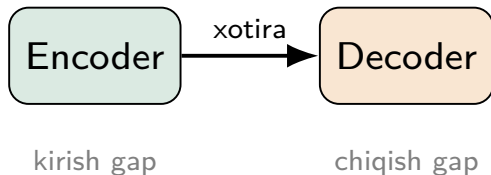
```
abs(f_t - 0.818) < 1e-3
```

Keyingi dars: xotiradan matn generatsiyasiga (Seq2Seq)

Endi bizda **xotirali model** bor (GRU/LSTM). Demak ketma-ketlikni faqat *tasniflash* emas, balki **generatsiya** ham qila olamiz:

- Matnni so'zma-so'z **yaratish** (til generatsiyasi).
- Ketma-ketlikdan ketma-ketlikka (*Seq2Seq*): kirish gapni chiqish gapga — masalan tarjima.
- Encoder → Decoder: bir LSTM o'qiydi, ikkinchisi yozadi.

9-ma'ruza: matn generatsiyasi va **Seq2Seq**. Keyinroq attention/Transformer (*kelajak mavzu*) — lekin avval xotirali dekoder bilan boshlaymiz.

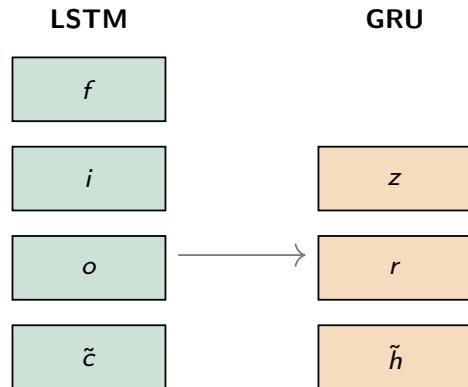


- ❶ Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- ❷ Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. *EMNLP*.
- ❸ Chung, J., et al. (2014). Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling. *arXiv:1412.3555*.
- ❹ Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3-nashr loyihasi), 9-bob: RNN va LSTM.
- ❺ PyTorch hujjatlari: `torch.nn.LSTM`, `torch.nn.GRU`.

Ilova S1: GRU = soddalashtirilgan LSTM

GRU ni LSTM ning ixchamlashtirilgan ko'rinishi deb qarash mumkin.

- c_t va h_t bitta holatga birlashtirilgan.
- forget + input $\Rightarrow$ yagona yangilash darvozasi z_t .
- 3 blok ($z, r, \tilde{h}$) vs LSTM ning 4 bloki.



4 $\rightarrow$ 3 blok

Ilova S2: forget darvozasini “ochiq” boshlash

Amaliy maslahat: forget bias ni $b_f = +1$ yoki $+2$ qilib boshlang.

Natija: $\sigma(\text{katta}) \approx 1$, ya'ni model boshida xotirani deyarli to'liq saqlaydi va keyin unutishni o'zi o'rganadi.

Manba: Jozefowicz va boshqalar (2015) — forget bias musbat bo'lishi o'qishni barqarorlashtiradi.

